



DATABASES FINAL PROJECT

Restaurant Ordering System (ROS)

A Multi-Tenant PostgreSQL Database for Small Restaurant Chains



31 Tables



RLS Multi-Tenancy



ACID Compliant



FastAPI



PostgreSQL 16



Next.js

The Data Fragmentation Crisis



Disconnected Systems

Small restaurant chains juggle separate platforms for orders, inventory, and loyalty programs, creating data silos and operational inefficiencies.



Multi-Tenant Isolation

Building SaaS for restaurants requires strict data isolation—Branch A must never access Branch B's orders, inventory, or customer data.



Consistency Challenges

Without atomic transactions, concurrent orders risk overselling inventory, leading to stock discrepancies and customer dissatisfaction.

Current Pain Points

- 1 Manual data reconciliation between systems
- 2 No real-time inventory visibility
- 3 Fragmented customer loyalty tracking
- 4 Security risks in multi-branch access

"We need a unified system that guarantees data isolation while maintaining ACID compliance across all operations."

Centralized PostgreSQL Architecture

Unified Data Platform

A single PostgreSQL 16 database consolidates orders, inventory, and loyalty into one ACID-compliant system, eliminating data fragmentation and ensuring real-time consistency across all restaurant operations.



Orders

Real-time processing



Inventory

Atomic deductions



Loyalty

LTV calculations

Row-Level Security (RLS)

Multi-tenancy is enforced at the database level using PostgreSQL RLS policies. Each branch operates within its own data sandbox, with 18 tables protected by automated security rules that prevent cross-tenant data leakage.

Branch A



Isolated data access

Branch B



Isolated data access

Tech Stack



PostgreSQL 16

Primary database



FastAPI

Backend framework



Next.js

Frontend framework

31

Tables

18

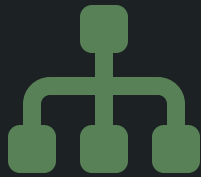
RLS Tables

100%

ACID

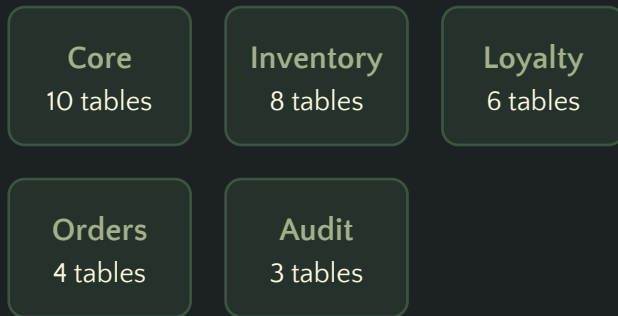
Schema Design & ERD

Entity Relationship Diagram



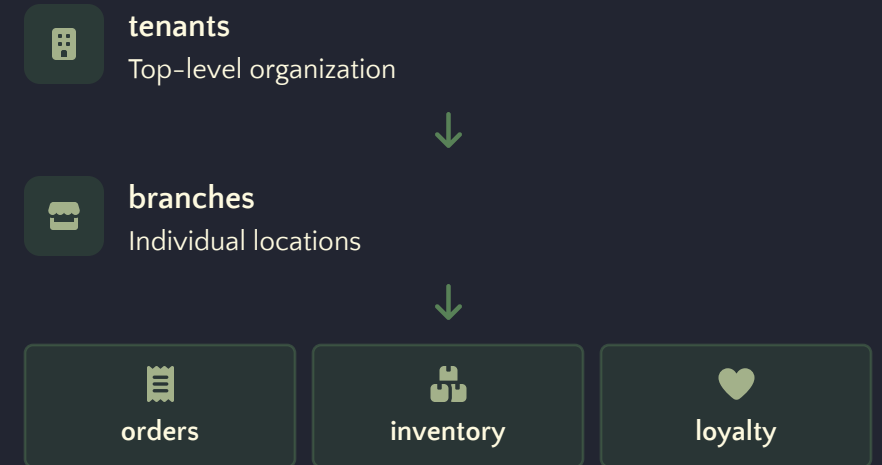
31 Tables

Multi-schema relational design



Visual: Place detailed ERD image here showing all 31 tables and relationships

Core Schema Hierarchy



Key Design Principles

- ✓ 3rd Normal Form (3NF)
- ✓ Foreign key constraints
- ✓ CHECK constraints
- ✓ Partitioned orders table

Normalization & Constraints

3rd Normal Form (3NF)

The schema strictly follows 3NF to eliminate data redundancy and ensure data integrity. Each non-key attribute depends only on the primary key, preventing update anomalies.

3NF Compliance Benefits

- 1 No repeating groups (1NF)
- 2 No partial dependencies (2NF)
- 3 No transitive dependencies (3NF)

Foreign Key Strategy

Heavy use of Foreign Keys with strategic ON DELETE behaviors to maintain referential integrity across the multi-tenant schema.

CASCADE

Auto-delete child records

RESTRICT

Prevent parent deletion

CHECK Constraints

Domain integrity is enforced through CHECK constraints that validate data at the database level, preventing invalid states before they occur.

Constraint Examples

```
quantity >= 0
Prevent negative inventory
```

```
price >= 0
Ensure valid pricing
```

```
status IN ('active', 'inactive')
Enum validation
```

Data Integrity Impact

100%
Constraint coverage

0
Data anomalies

Indexing, Partitions & RLS

Indexing Strategy

B-Tree Indexes

Foreign keys and search columns
O(log n) lookup performance

GIN Indexes

JSONB settings and arrays
Efficient containment queries

BRIN Indexes

Partitioned orders table
Block-range minimal storage

Index Performance

Query speed	10x faster
Storage overhead	< 5%

Table Partitioning

The `orders` table is partitioned by month on `created_at`, enabling efficient query performance and simplified data management at scale.

Partition Structure

- orders_2024_01
- orders_2024_02
- orders_2024_03
- ... Auto-created monthly

50%

Faster queries

80%

Easier archiving

Row-Level Security

RLS policies are applied to 18 tables, automatically filtering data based on the current tenant context, ensuring complete data isolation.

RLS Coverage

Protected tables	18 / 31
Policy type	USING
Enforcement	Automatic

 Every query automatically filters by tenant_id

Audit Triggers

All UPDATE and DELETE operations are automatically logged with timestamps, user context, and before/after values for complete audit trails.

Atomic Transactions: fn_place_order

</>Stored Procedure Logic

```
-- fn_place_order procedure
BEGINTRANSACTION;
  -- 1. Lock inventory rows
  SELECT ... FOR UPDATE;
  -- 2. Check stock availability
  IF stock >= quantity THEN
    -- 3. Deduct ingredients
    UPDATE inventory ...;
    -- 4. Create order record
    INSERT INTO orders ...;
    -- 5. Update loyalty points
    UPDATE loyalty_accounts ...;
  END IF;
COMMIT;
```

Visual: Show complete PL/pgSQL code with syntax highlighting

ACID Compliance



Atomic
All or nothing



Consistent
Valid state only



Isolated
No interference



Durable
Persistent

Concurrency Handling

The **FOR UPDATE** lock prevents race conditions when multiple branches place orders simultaneously, ensuring accurate inventory deductions.

Transaction Flow

- 1 Lock inventory rows
- 2 Validate stock levels
- 3 Execute deductions
- 4 Commit or rollback

Materialized Views & Query Results

mv_customer_ltv

Materialized view that calculates Customer Lifetime Value by aggregating total orders, average order value, and purchase frequency per customer.

```
-- Customer LTV Query
SELECT customer_id,
       SUM(total_amount) AS ltv,
       COUNT(*) AS order_count,
       AVG(total_amount) AS avg_order
FROM orders
GROUP BY customer_id;
```

Sample Results

Customer	LTV	Orders
Customer A	\$1,240	24
Customer B	\$890	18
Customer C	\$2,150	31

mv_daily_revenue

Aggregates daily revenue by branch, enabling real-time business intelligence dashboards and financial reporting with automatic refresh.

```
-- Daily Revenue Query
SELECT
  DATE(created_at) AS day,
  branch_id,
  SUM(total_amount) AS revenue,
  COUNT(*) AS orders
FROM orders
GROUP BY day, branch_id;
```

Sample Results

Date	Branch	Revenue
2024-01-15	Downtown	\$3,450
2024-01-15	Midtown	\$2,890
2024-01-16	Downtown	\$4,120

100x

Faster analytics

5min

Refresh interval

Zero

Query impact